Introduction to Blockchain

CSE-355

Ahmed Akhtar
Department of Computer Science
SMCS, IBA Karachi
Spring 2026

Lecture 19
Tuesday, Apr 7, 2026

Reducing Bitcoin Transaction Fee

- Decrease transaction size
  - Direct reduction in fees
  - SegWit, Taproot, Schnor signature helped achieve this goal
- Increase block size
  - Block fits more transactions so helps reduce fee indirectly
  - SegWit increased block size as a soft fork
  - Introduced discount on witness data, effectively increasing block size from 1 MB to about 4 MB
- Batch Processing of transactions
  - Fee is divided across multiple transactions
  - Easier for custodial wallets (that hold private keys of multiple users)
  - Difficult for non-custodial wallets for different users

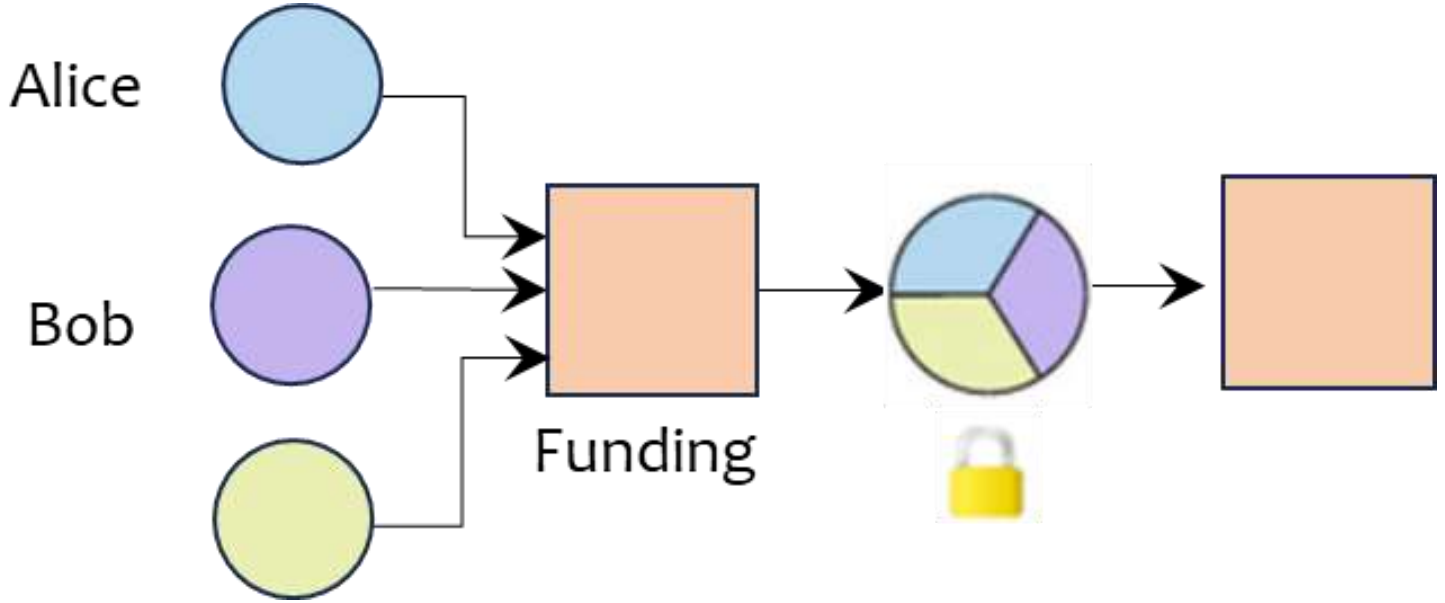
Transaction Batch Processing

- There are two transaction batch processing models
  - Collaborative batching via joint transaction creation
  - Non-collaborative batching with independently signed transactions
- Collaborative batching:
  - A joint single transaction is created with multiple inputs and outputs
  - As single transaction serves many users, transaction fee is shared among all of them
  - Exchanges & custodial wallets with access to all the private keys can implement this model easily
  - Wallet level improvement whenever possible and no consensus level changes are required
- Non-collaborative batching:
  - Not possible in Bitcoin
  - UTXO model prevents combining independently signed transactions
  - Any attempt to join or merge transactions would invalidate their signatures
  - Once a transaction enters mempool, it becomes atomic, i.e., it must be included as it is in the block
- In contrast, it is possible to build non-collaborative batching solutions in Ethereum, allowing significant fee reduction (these solutions are popularly known as roll ups)

Channel Factory

- A payment channel is opened with a 2-of-2 on-chain multisig funding transaction
- The fee can be further reduced if multiple users jointly fund a single on-chain transaction and then open several payment channels

Example: Alice, Bob, and Charlie create one funding transaction and then open three payment channels (A–B, A–C, B–C) using the same funding transaction



This setup where multiple parties deposit their funds into a single on-chain funding transaction is known as a channel factory in Bitcoin community

Channel Factory

- Channel factory is an appealing idea and seems a natural extension of the payment channels
- However, there are several practical challenges
  - The problem has a combinatorial complexity (N choose 2)
  - Each participant must track commitment states for every internal channel they are part of
  - Adding or removing parties or uncooperative peers often requires on-chain reconfiguration or complex protocols
  - Operational complexity, dispute handling, and watchtower needs raise costs

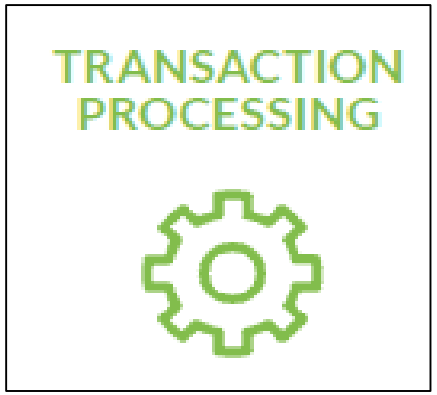
Channel factories represent the next frontier in Bitcoin scalability, calling for new designs and new research

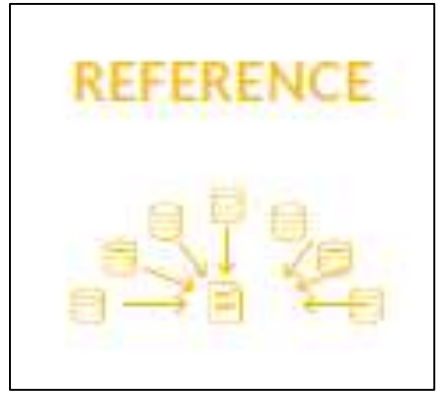
Satoshi Nakamoto realized his vision of blockchain by launching Bitcoin in 2009

Since then, several alternative blockchains with varying features have been developed

Blockchain Design Considerations

- A wide variety of blockchains have been designed
- However, there are some important design considerations that require careful attention

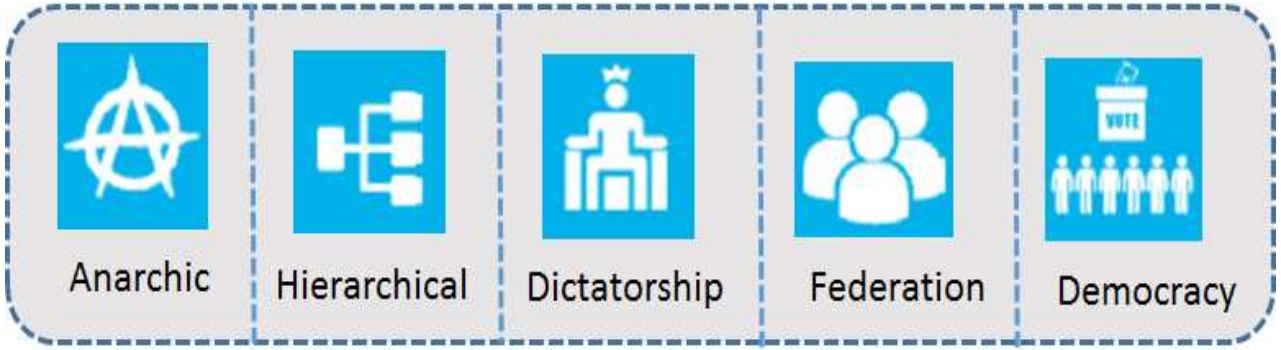




Together, these considerations capture the organizational, technical, economic, and architectural dimensions of the blockchain design

Protocol Governance

Protocol governance refers to the mechanisms and processes by which decisions are made regarding the rules, protocols, and upgrades of a blockchain network



- Anarchic: Governance norms, processes, and procedures are inherited from free/open-source software community
- Hierarchical: There is a recognized leadership e.g., foundation or committee to guide/shape the decision making
- Dictatorship: Decisions are taken by a single entity e.g., person, company, mining pool
- Federation: A group of agents have the power to vote on alterations
- Democratic: Protocol change proposals are voted on, with each vote weighted by the importance of each voter

Implications:

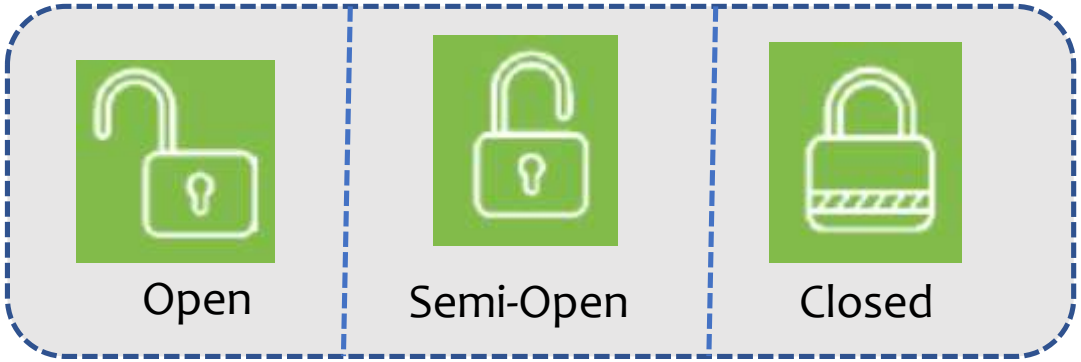
- Transparency
- Efficiency
- Perceived legitimacy

Bitcoin governance model?

Anarchic

Network Access

Network access determines the right of entry to the system



- Open: Unrestricted network access where anyone can freely join, leave, rejoin
  - Joining simply requires downloading and running a software client (there are no gatekeepers)
- Semi-open: Access is partially restricted where prospective participants need to apply
  - Decided via on-chain voting/approval of existing network participants
- Closed: Access is restricted and decided by a gatekeeper

Implications:

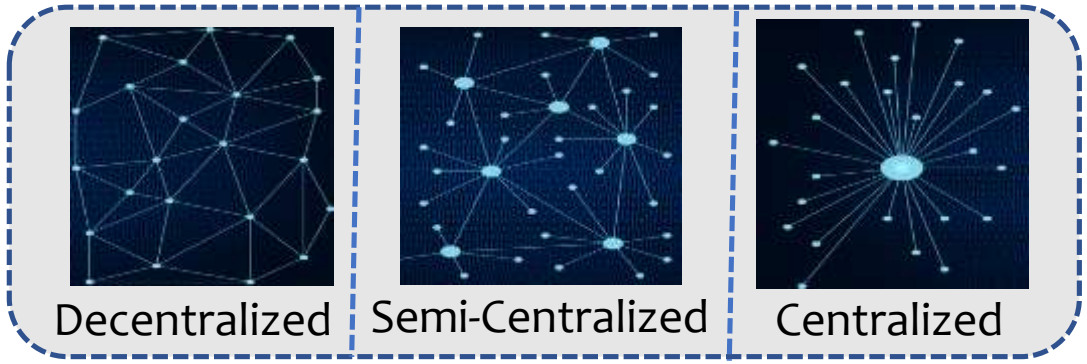
- Diversity of participants
- Consensus mechanism
- Trust requirements

Bitcoin Network Access Model?

Open Access

Transaction Processing

Transaction processing refers to the degree of centralization in terms of selecting transactions and adding records to the global ledger



- Decentralized: Any network participant has the right to participate in transaction processing
- Semi-Centralized: A select subset of network participants have the *authorization* to process transactions
- Centralized: A single participant has control over transaction processing

Implications:

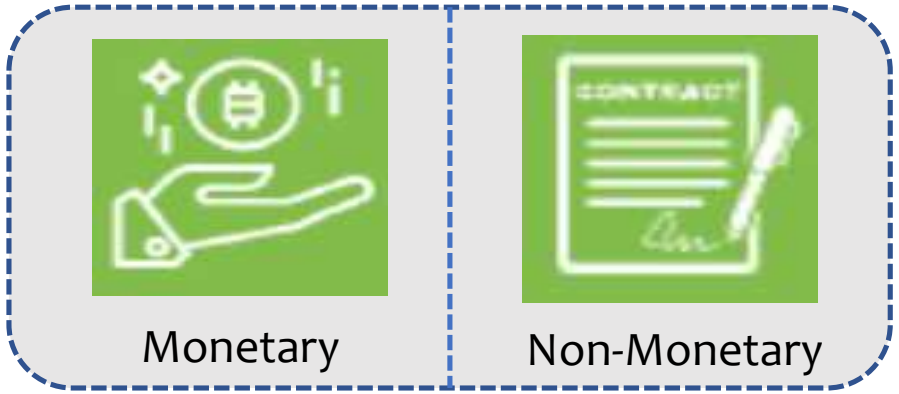
- Transaction finality
- Participation
- Degree of tamper resistance

Bitcoin Transaction Processing Model?

Decentralized

Incentives

Incentives motivate the nodes to participate in various activities and to validate and process transactions



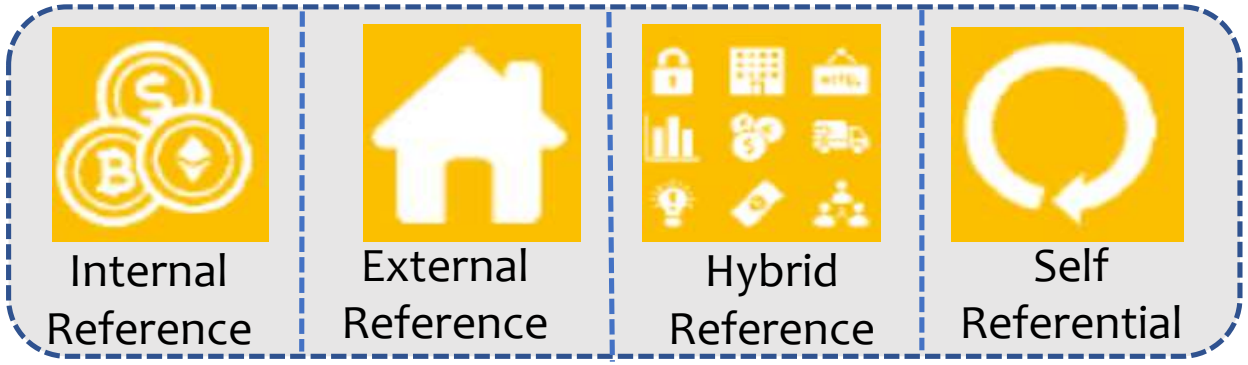
Implications:
- Network type (open/close)
- Security
- Viability

- Monetary incentives: Block rewards plus transaction fees
  - These are intrinsic rewards generally paid in the native cryptocurrency of the system
- Non-monetary incentives: These incentive are in the form of contractual obligations or reputations
  - These rewards are extrinsic rewards

Bitcoin Incentives Model?
Monetary

Reference Model

Reference model establishes whether the data produced by participants references internal or external information



Implications:
- Smart Contracts

- Internal reference: Tracks information about variables that are native to the system
- External reference: Refers to data that tracks information about variables that exist outside of the system
- Hybrid reference: Supports both internal and external reference
- Self-Referential: Incorporates references to its own components and processes, enabling self-awareness
  - Useful for smart contract implementation

Bitcoin Reference Model?
Internal Reference

Bitcoin

		Bitcoin
Protocol Governance	Anarchic	✓
	Hierarchical	
	Dictatorship	
	Federation	
	Democracy	
Network Access	Open	✓
	Semi-open	
	Closed	
Transaction Processing	Decentralized	✓
	Semi-centralized	
	Centralized	
Incentives	Intrinsic	✓
	Extrinsic	
Reference Model	Internal variable/states	✓
	External variable/states	
	Hybrid	
	Self-Referential	

Lecture 20
Thursday, Apr 9, 2026

Origins of Ethereum

- Vitalik Buterin – Born in 1994
- Founded Bitcoin magazine in 2012
- Argued for a more elaborate scripting language for Bitcoin
- Wrote a whitepaper detailing his Ethereum ideas in 2013
- Developed his ideas into a working blockchain
- First Ethereum block was mined on July 30, 2015

- One of the most influential persons in the blockchain community
- Frequently writes on various blockchain topics https://vitalik.ca/

Ethereum

- Purpose: General purpose blockchain
- Network launch: July 2015 (Ethereum Foundation)
- Value proposition: Smart contract functionality on the blockchain to enable the creation of automated and secure agreements and transactions

Ethereum is Turing Complete

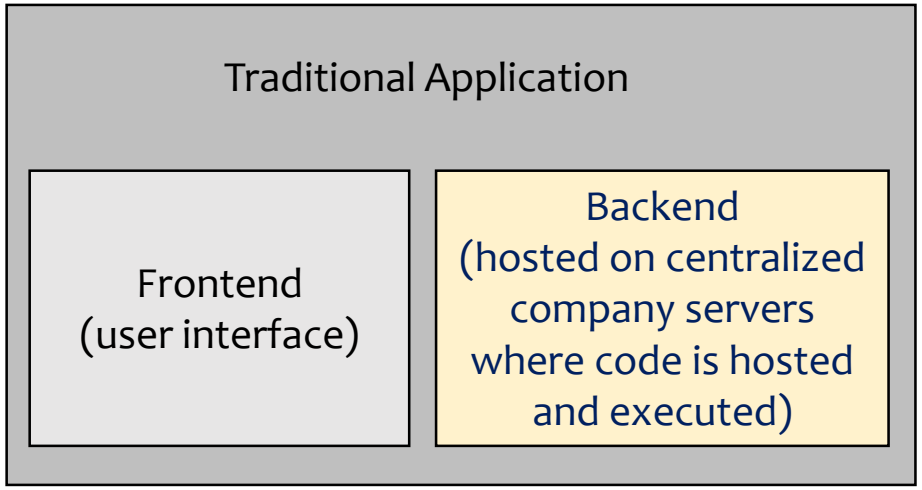
- Ethereum is designed to write smart contracts capable of executing code with arbitrary and unbounded complexity
- In more technical terms, Ethereum is Turing-complete, i.e., it can theoretically compute anything that can be computed
  - Turing-complete programming languages: C++/Python
  - Non-Turing complete programming language: Bitcoin scripting language
- Ethereum smart contracts are mostly written in Solidity and Vyper
  - Solidity's syntax is similar to C++
  - Vyper's syntax is similar to Python

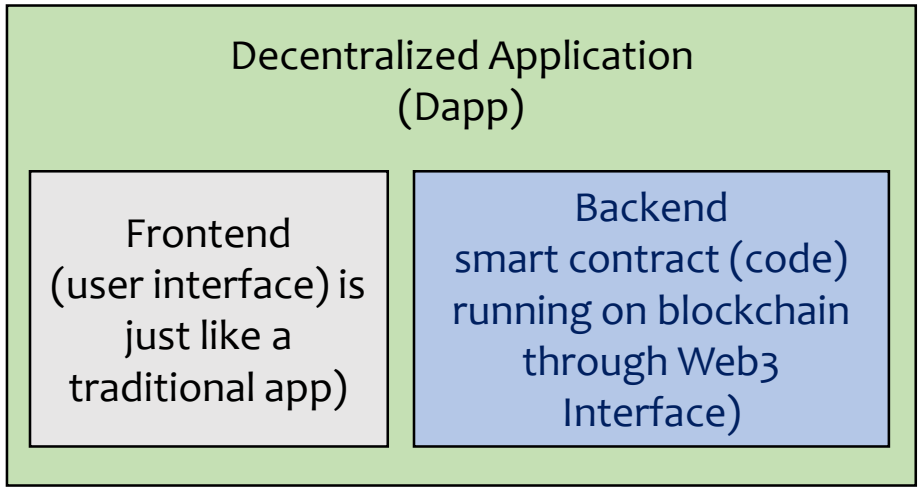
Smart Contract

- Several steps are involved in creating and deploying a smart contract on Ethereum blockchain
- Step 1: Write a smart contract in a high-level programming language
  - Solidity is the language of choice
- Step 2: Compile the smart contract
  - Solidity compiler comes as a standalone executable or bundled in Integrated Development Environments (IDEs)
- Step 3: Deploy the smart contract on blockchain
  - Smart contract can only be deployed by some user account, also called externally owned account (EOA)
- Steps 1 & 2 require programming i.e., writing a code and compiling it to check for any errors or warnings
- Step 3 involves some new blockchain concepts

Decentralized Applications

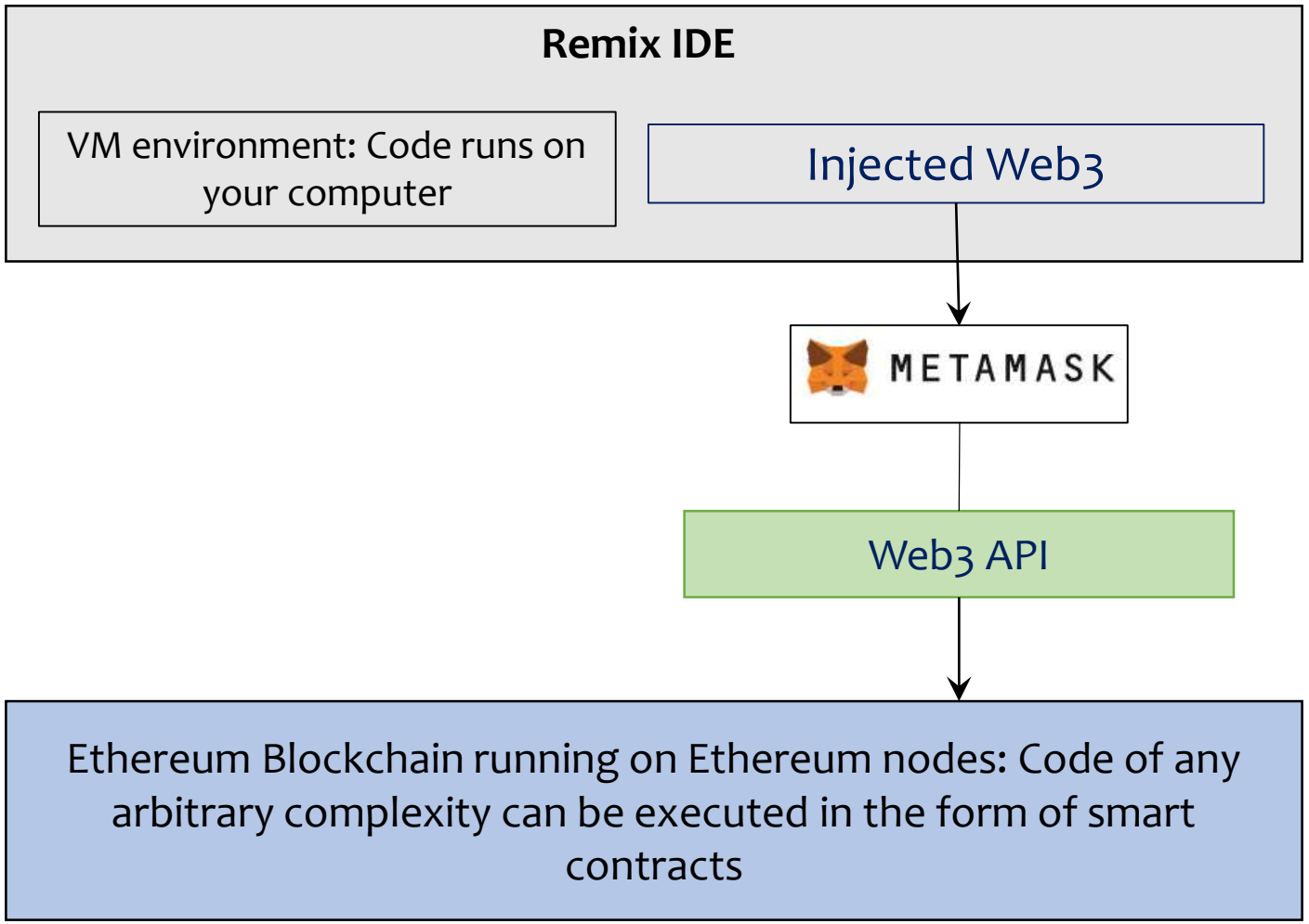
Apps have a frontend (user interface) and a backend





Mostly, games, NFT swaps, Metaverse, decentralized finance (DeFi), wallets that deploy smart contracts (MetaMask)

Web3



Web3 (APIs, libraries, protocols, tools) is the collective term for the bridge between the frontend and the blockchain, allowing users to interact with smart contracts directly through their wallets, without any centralized server in between

Benefits of DApps

- Resistance to censorship
  - No single entity on the network can block users from submitting transactions and deploying DApp
- Data integrity
  - Data on blockchain is immutable and therefore, malicious actors cannot forge transactions or data already publicly available
- Trustable computation/verifiable behavior
  - Smart contracts are guaranteed to execute in predictable ways
- Zero downtime
  - Backend is on blockchain that is always running
- Privacy
  - Real world identity of DApp users remains hidden

https://ethereum.org/en/developers/docs/dapps/

Drawbacks of DApps

- Maintenance
  - DApps can be harder to maintain (updates or bug fixes) because the code and data published to the blockchain cannot be modified
- Performance overhead
  - There is a huge performance overhead because scaling is hard due to decentralization and proof of work
- Network congestion
  - Transaction throughput of public blockchains is very low (Ethereum network can only process 15 to 30 tps) and therefore the pool of unconfirmed transactions can quickly balloon up
- Centralization Tendencies
  - User & developer friendly solutions may store lot of data and sensitive information on server-side, which may eliminate most of the blockchain advantages

Ethereum

		Bitcoin	Ethereum
Protocol Governance	Anarchic	✓	
	Hierarchical		✓
	Dictatorship		
	Federation		
	Democracy		
Network Access	Open	✓	✓
	Semi-open		
	Closed		
Transaction Processing	Decentralized	✓	✓
	Semi-centralized		
	Centralized		
Incentives	Intrinsic	✓	✓
	Extrinsic		
Reference Model	Internal variable/states	✓	
	External variable/states		
	Hybrid		✓
	Self-Referential		✓